tôi muốn công cụ kiểm tra dịch vụ được sử dụng để kiểm tra xem dịch vụ có cung cấp được cho người dùng ở ngoài internet được không, việc kiểm tra được thực hiện theo flow như sau: 1. Kiểm tra trạng thái dịch vụ tại máy chủ 2. Kiểm tra port service 3. kiểm tra firewall local trong server 4. kiểm tra firewall bên ngoài 5. Test dịch vụ có tới được người dùng ở internet hay không Đầu vào của tool sẽ gồm tên dịch vụ, cổng dịch vụ, cổng truy cập từ internet vào firewall, địa chỉ ip của firewall ngoài internet để truy cập, loại giao thức để kiểm tra kết nối (TCP/UDP/ICMP). Đầu ra sẽ là một file ghi lại các log kiểm tra lần lượt các bước có thể bao gồm việc theo dõi gói tin, traceroute để xem path. Tool trên được viết dưới dạng một scripts Python và được đặt trong server linux

Bạn đang mô tả một **network service reachability diagnostic tool** – mục tiêu là xác định dịch vụ có thực sự public ra Internet hay bị chặn ở tầng nào (service / socket / local firewall / perimeter firewall / routing / NAT).

Tôi sẽ thiết kế cho bạn:

- ✅ Kiến trúc logic theo đúng flow bạn yêu cầu
- ✅ Thiết kế tham số đầu vào
- ✅ Chiến lược kiểm tra theo từng layer (OSI perspective)
- ✅ Skeleton Python script hoàn chỉnh
- ✅ Định dạng log output
- ✅ Gợi ý mở rộng (tcpdump tracking, traceroute path analysis)

I. Flow kiểm tra tổng thể

```
[1] Systemd service state
        ↓
[2] Listening socket (ss/netstat)
        ↓
[3] Local firewall (iptables/nftables/ufw)
        ↓
[4] External firewall/NAT test (port exposure)
        ↓
[5] Real Internet test (external probe + traceroute)
```

Phân loại theo tầng:

Bước	Layer	Mục đích
1	Application	Service có chạy không
2	Transport	Có bind port không
3	Host firewall	Local OS có block không
4	Network perimeter	Firewall/NAT có forward không

5	End-to-end	Internet có reach được không

II. Input của tool

Ví dụ:

```
python service_check.py \
  --service nginx \
  --service-port 80 \
  --public-port 8080 \
  --firewall-ip 203.113.x.x \
  --protocol tcp
```

Tham số

Tham số	Ý nghĩa
service	tên service systemd
service_port	port chạy nội bộ
public_port	port được NAT ra ngoài
firewall_ip	IP public firewall
protocol	tcp / udp / icmp

III. Thiết kế kiểm tra từng bước

1⃣ Kiểm tra trạng thái service

Dùng:

```
systemctl is-active <service>
```

Hoặc:

```
systemctl status <service>
```

Nếu inactive → STOP ngay.

2⃣ Kiểm tra port có listen không

TCP:

```
ss -lntp | grep :PORT
```

UDP:

```
ss -lnup | grep :PORT
```

Nếu không thấy LISTEN → cấu hình sai.

3⃣ Kiểm tra firewall local

Tuỳ hệ thống:

iptables

```
iptables -L -n -v
```

Check rule:

- DROP?
- REJECT?
- ACCEPT?

nftables

```
nft list ruleset
```

ufw

```
ufw status verbose
```

4⃣ Kiểm tra firewall ngoài (NAT / Port Forward)

Test từ chính server ra public IP:

```
nc -vz firewall_ip public_port
```

Hoặc:

```
telnet firewall_ip public_port
```

Nếu fail → NAT/Firewall ngoài chưa forward.

5⃣ Test từ Internet (real reachability)

Cách chuyên nghiệp:

A. TCP connect test

```
socket.create_connection()
```

B. UDP test

Send UDP packet → chờ response

C. ICMP test

```
ping firewall_ip
```

D. Traceroute

```
traceroute firewall_ip
```

E. Packet capture (optional)

```
tcpdump -i any port PORT
```

Để xem SYN có vào server không.

IV. Kiến trúc log output

File log:

```
service_check_20260302_070000.log
```

Ví dụ nội dung:

```
[STEP 1] Checking service status...
Service nginx: ACTIVE

[STEP 2] Checking listening port...
Port 80 LISTENING on 0.0.0.0

[STEP 3] Checking local firewall...
iptables: ACCEPT rule found

[STEP 4] Checking public NAT...
Connection to 203.113.x.x:8080 SUCCESS

[STEP 5] Internet reachability test...
TCP handshake SUCCESS
Traceroute:
1 192.168.1.1
2 10.10.10.1
3 203.113.x.x
```

V. Python Script Skeleton

Đây là bản production-ready cơ bản:

```
#!/usr/bin/env python3

import subprocess
import socket
import argparse
import datetime
import sys
```

```
def run_cmd(cmd):
    try:
        result = subprocess.run(cmd, shell=True, capture_output=True,
text=True)
        return result.stdout + result.stderr
    except Exception as e:
        return str(e)

def log_write(logfile, text):
    logfile.write(text + "\n")
    print(text)

def check_service(service, logfile):
    log_write(logfile, "[STEP 1] Checking service status...")
    output = run_cmd(f"systemctl is-active {service}")
    log_write(logfile, output.strip())
    return "active" in output

def check_port(port, protocol, logfile):
    log_write(logfile, "[STEP 2] Checking listening port...")
    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"
    output = run_cmd(cmd)
    log_write(logfile, output if output else "Port not listening")
    return bool(output)

def check_local_firewall(logfile):
    log_write(logfile, "[STEP 3] Checking local firewall (iptables)...")
    output = run_cmd("iptables -L -n -v")
    log_write(logfile, output[:1000])
    return True

def test_public_connect(ip, port, protocol, logfile):
    log_write(logfile, "[STEP 4 & 5] Testing public connectivity...")
    try:
        if protocol.lower() == "tcp":
            sock = socket.create_connection((ip, port), timeout=5)
            sock.close()
            log_write(logfile, "TCP Connection SUCCESS")
            return True
        elif protocol.lower() == "udp":
            sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            sock.sendto(b"test", (ip, port))
            log_write(logfile, "UDP Packet sent (no guarantee of response)")
            return True
        elif protocol.lower() == "icmp":
            output = run_cmd(f"ping -c 4 {ip}")
```

```
            log_write(logfile, output)
            return "0% packet loss" in output
    except Exception as e:
        log_write(logfile, f"Connection FAILED: {e}")
        return False

def traceroute(ip, logfile):
    log_write(logfile, "[Traceroute]")
    output = run_cmd(f"traceroute {ip}")
    log_write(logfile, output)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--service", required=True)
    parser.add_argument("--service-port", type=int, required=True)
    parser.add_argument("--public-port", type=int, required=True)
    parser.add_argument("--firewall-ip", required=True)
    parser.add_argument("--protocol", required=True)
    args = parser.parse_args()

    log_filename =
f"service_check_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"
    with open(log_filename, "w") as logfile:

        if not check_service(args.service, logfile):
            log_write(logfile, "Service is NOT running. STOP.")
            sys.exit(1)

        if not check_port(args.service_port, args.protocol, logfile):
            log_write(logfile, "Service port not listening. STOP.")
            sys.exit(1)

        check_local_firewall(logfile)

        test_public_connect(args.firewall_ip, args.public_port,
args.protocol, logfile)

        traceroute(args.firewall_ip, logfile)

    print(f"\nLog saved to {log_filename}")

if __name__ == "__main__":
    main()
```

VI. Cách dùng thực tế

Ví dụ kiểm tra nginx:

```
python3 service_check.py \
  --service nginx \
  --service-port 80 \
  --public-port 8080 \
  --firewall-ip 203.113.10.5 \
  --protocol tcp
```

VII. Nâng cấp chuyên sâu (khuyên dùng)

Nếu bạn muốn tool mang tính chuyên nghiệp (SOC / NOC level), nên bổ sung:

✔ JSON structured log

✔ Tự detect iptables vs nft

✔ tcpdump auto capture 5 giây

✔ Check routing table

```
ip route
```

✔ Check default gateway

✔ Check NAT hairpin

✔ Kiểm tra SELinux/AppArmor

✔ Export HTML report

VIII. Kiến trúc nâng cao (Enterprise version)

Bạn có thể chia tool thành 3 module:

```
core/
  service.py
  socket_check.py
  firewall.py
  network.py
report/
  html_report.py
main.py
```

IX. Kết luận

Tool của bạn sẽ:

- Phân tích theo layer
- Xác định chính xác chặn ở đâu
- Có log forensic
- Có traceroute path
- Có test TCP/UDP/ICMP

Nếu bạn muốn, tôi có thể viết cho bạn phiên bản:

- 🔥 Production enterprise version
- 🔥 Có parallel test
- 🔥 Có color output
- 🔥 Có JSON + HTML export
- 🔥 Có packet capture tự động

Bạn muốn nâng cấp theo hướng lab học tập hay triển khai thực tế production?

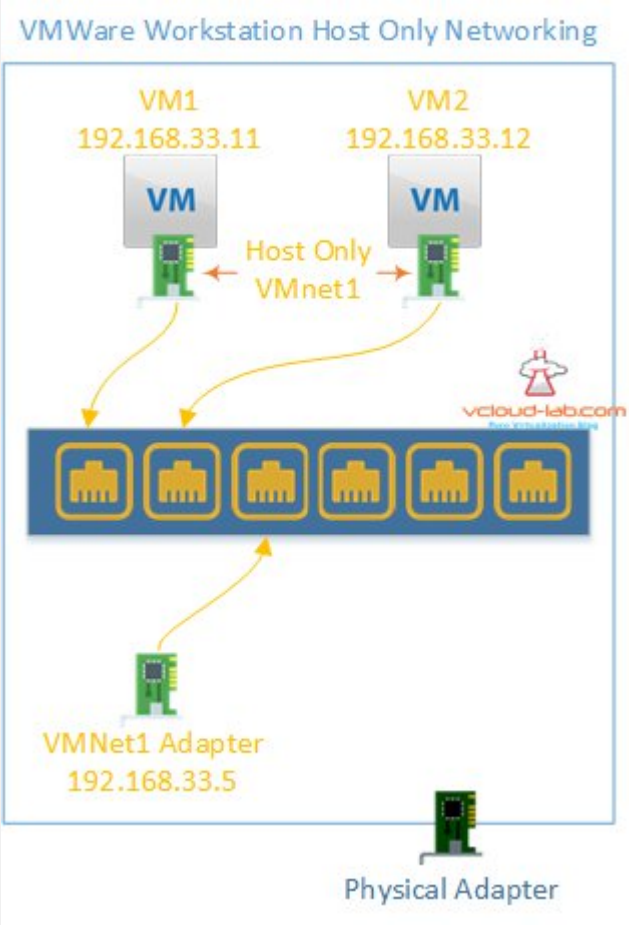
tool trên được thực hiện trên môi trường giả lập VM

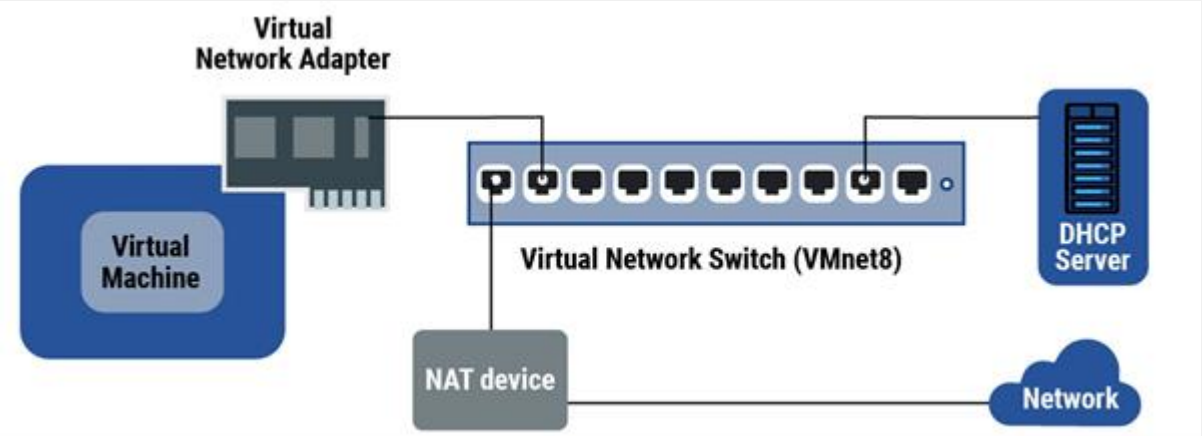
Rõ ràng. Nếu tool chạy trong **môi trường VM lab (VMware / VirtualBox / EVE-NG / GNS3)** thì có một số yếu tố kiến trúc mạng ảo cần xử lý khác so với môi trường production thật.

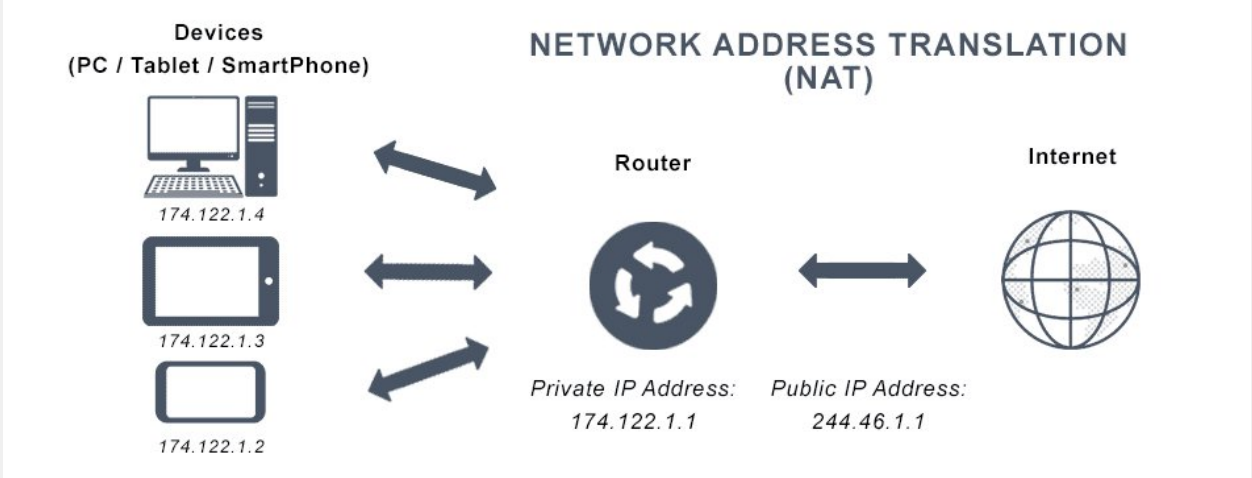
Tôi sẽ phân tích theo hướng kỹ thuật triển khai trong lab.

I. Các topology VM phổ biến

☐VMware Workstation – NAT mode







4

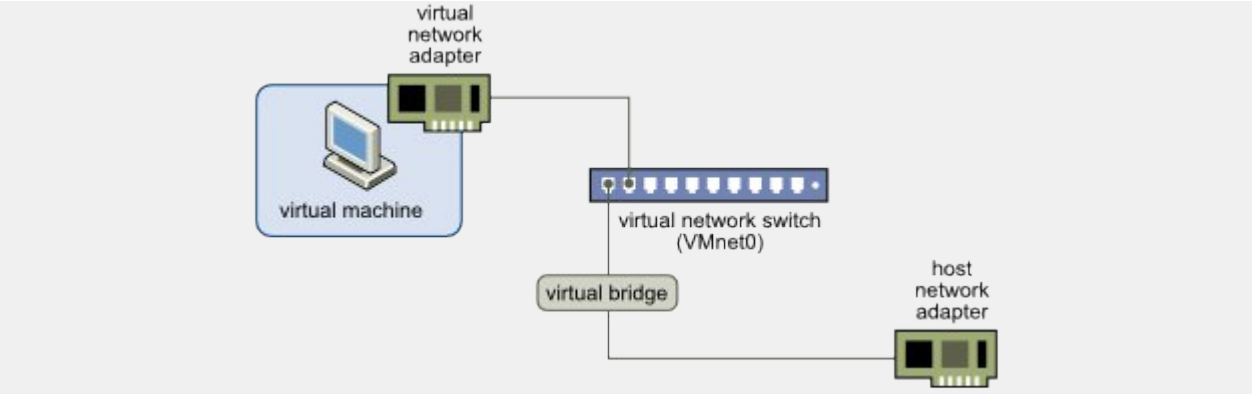
Đặc điểm:

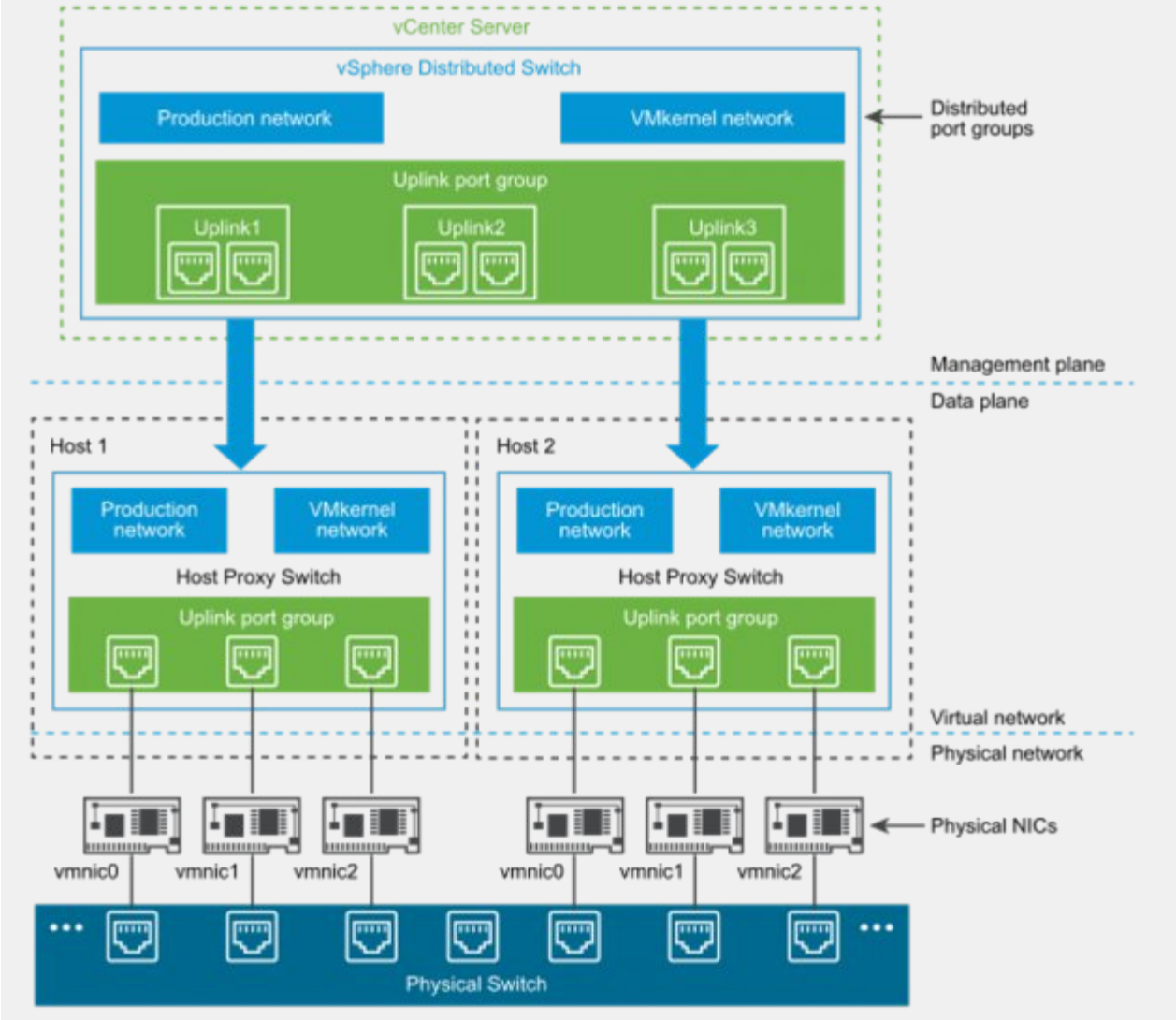
- VM có IP private (ví dụ 192.168.200.0/24)
- VMware NAT engine làm gateway
- Không có Internet inbound thực sự
- Port forward phải cấu hình trong Virtual Network Editor

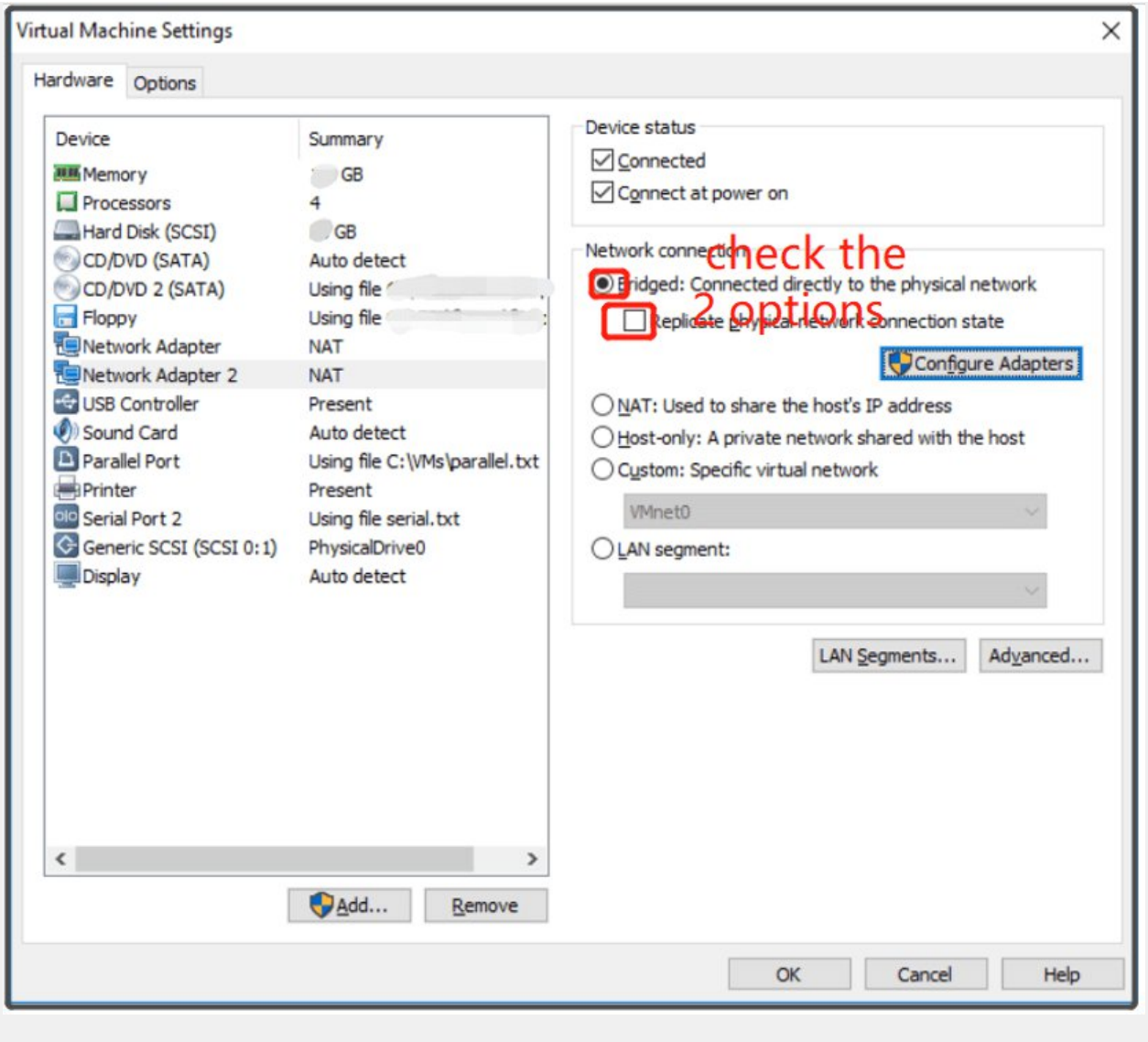
⚠ Trong mode này:

- Bước 4 (external firewall test) có thể fail do NAT 2 lớp
- Traceroute sẽ dừng tại VMnet8 gateway

2⃣ VMware – Bridged mode




vCenter Server
vSphere Distributed Switch
Production network
VMkernel network
Distributed port groups
Uplink port group
Uplink1
Uplink2
Uplink3
Management plane
Data plane
Host 1
Host 2
Production network
VMkernel network
Host Proxy Switch
Uplink port group
Virtual network
Physical network
Physical NICs
vmnic0
vmnic1
vmnic2
Physical Switch



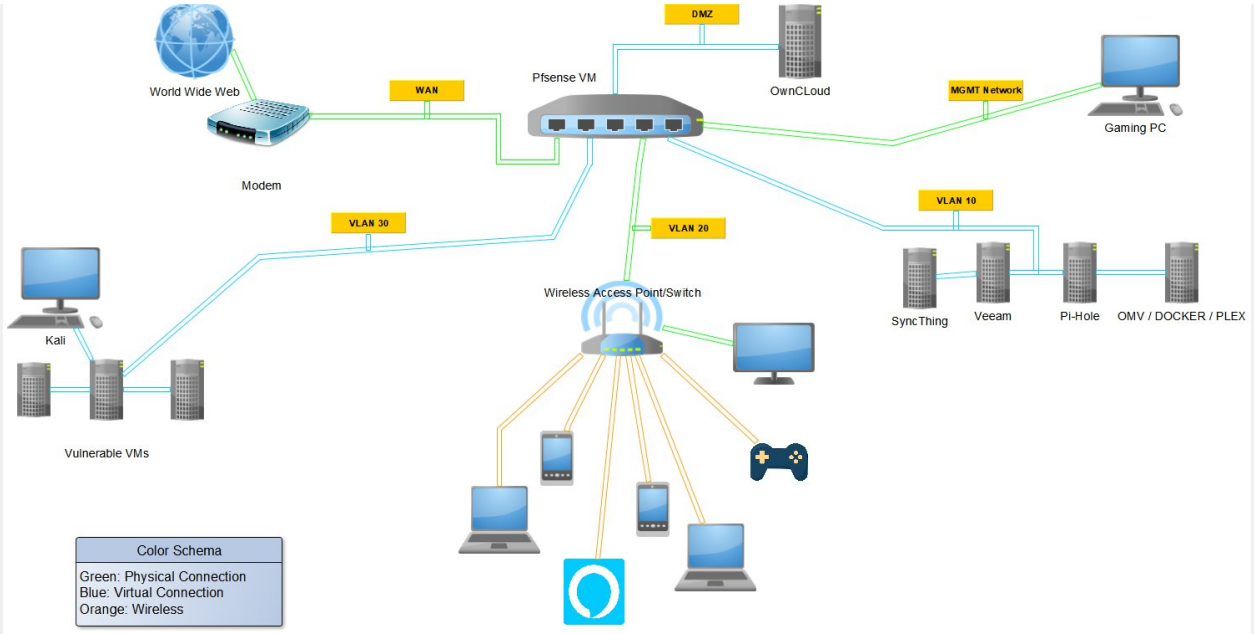
4

Đặc điểm:

- VM nằm cùng subnet với mạng thật
- Có IP LAN thật
- Có thể forward port từ router thật

→ Đây là mode lý tưởng để test tool.

3. Lab có Firewall VM (ví dụ pfSense)



4

Flow sẽ như sau:

```
Client Internet giả lập
        ↓
Firewall (pfSense WAN)
        ↓ NAT
Server VM (LAN)
```

Tool sẽ xác định chặn ở:

- Service
- Local firewall
- pfSense rule
- NAT rule
- Route

II. Điều chỉnh tool cho môi trường VM

☐Phát hiện đang chạy trong VM

Thêm hàm detect:

```
def detect_vm():
    try:
        output = subprocess.check_output("systemd-detect-virt", shell=True,
text=True)
```

```
        return output.strip()
    except:
        return "unknown"
```

Log ra:

```
Environment: vmware / kvm / oracle
```

2⃣ Kiểm tra thêm gateway layer (rất quan trọng trong VM)

Trong VM, lỗi thường nằm ở default gateway.

Thêm bước:

```
def check_gateway(logfile):
    output = run_cmd("ip route | grep default")
    log_write(logfile, "[Gateway Check]")
    log_write(logfile, output)
```

3⃣ Kiểm tra ARP resolution

Trong lab ảo nhiều khi không ARP được gateway:

```
run_cmd("ip neigh")
```

4⃣ Hairpin NAT problem (rất hay gặp trong lab)

Nếu test từ chính server tới firewall public IP → có thể fail do NAT reflection disabled.

pfSense cần bật:

```
System > Advanced > Firewall & NAT > NAT Reflection
```

Nếu không → tool sẽ báo FAIL nhưng thực tế Internet bên ngoài vẫn vào được.

III. Điều chỉnh bước 5 cho VM lab

Trong môi trường lab, không có Internet thật.

Vì vậy nên:

Cách 1 – Tạo 1 VM giả lập Internet

```
VM-Client (Internet)
        |
        WAN
      pfSense
        |
```

```
        LAN
    VM-Server
```

Tool khi test public IP nên thực hiện từ VM-Client thay vì từ server.

Cách 2 – Reverse connect test

Thay vì server connect ra public IP, dùng netcat ở VM-Client:

```
nc -l 8080
```

Server thử connect vào đó.

IV. Những lỗi thường gặp trong môi trường VM

Vấn đề	Triệu chứng
NAT 2 lớp	Traceroute dừng tại VMnet
Promiscuous mode off	Không thấy packet
Firewall Windows host chặn	Port forward fail
pfSense rule sai direction	SYN vào nhưng không reply
ICMP bị drop	Ping fail nhưng TCP vẫn OK
Checksum offloading	tcpdump thấy gói lỗi

V. Nâng cấp tool cho lab chuyên nghiệp

Tôi đề xuất thêm các chức năng sau cho môi trường giả lập:

✔ Kiểm tra network mode (NAT/Bridge)

```
ip addr
```

✔ Check route table

```
ip route
```

✔ Check NAT reflection

(Đối với pfSense → detect via curl API nếu có)

✔ Auto tcpdump capture 5 giây

```
subprocess.Popen("timeout 5 tcpdump -nn -i any port 80", shell=True)
```

VI. Flow tối ưu cho môi trường VM

```
[VM Detect]
        ↓
[Service]
        ↓
[Listening Port]
        ↓
[Local Firewall]
        ↓
[Routing Table]
        ↓
[Gateway Reachability]
        ↓
[NAT/Firewall VM]
        ↓
[External Client VM Test]
```

VII. Kết luận kỹ thuật

Nếu bạn triển khai trong lab VM:

- Nên dùng Bridged mode nếu muốn test thật
- Hoặc tạo 1 VM đóng vai Internet
- Phải xử lý hairpin NAT
- Tool cần thêm gateway + route + ARP check
- Traceroute trong NAT mode sẽ không phản ánh Internet thật

Nếu bạn cho tôi biết:

- Bạn đang dùng VMware hay VirtualBox?
- Có dùng pfSense VM không?
- Server và client nằm trong cùng host hay khác host?

Tôi có thể thiết kế lại tool tối ưu đúng topology lab của bạn.

lab được xây dựng như ảnh, viết lại tool để thực hiện kiểm tra dịch vụ, khi kiểm tra firewall bên ngoài sẽ kiểm tra xem gói tin có thể đi ra từ internet hay không

Theo sơ đồ bạn cung cấp, topology lab là:

```
Internet (192.168.232.0/24 - NAT/Bridged)
        |
WAN: 192.168.232.130
   pfSense
LAN: 10.10.10.3
        |
Server: 10.10.10.10
```

Yêu cầu quan trọng:

Khi kiểm tra firewall bên ngoài → phải xác định gói tin từ Internet có đi vào được pfSense WAN và xuống tới server hay không.

Trong môi trường này, kiểm tra đúng cách phải gồm:

- Kiểm tra service nội bộ
- Kiểm tra local firewall server
- Kiểm tra route ra gateway
- Kiểm tra NAT + rule trên pfSense

- Kiểm tra packet có vào WAN không (tcpdump trên WAN)
- Kiểm tra packet có vào LAN không (tcpdump trên LAN)
- Test từ phía “Internet side”

Dưới đây là phiên bản tool được viết lại phù hợp đúng topology của bạn.

🔎 Logic kiểm tra theo topology lab

STEP 1 – Kiểm tra service

systemctl is-active

STEP 2 – Kiểm tra port LISTEN

ss -lntp / ss -lnup

STEP 3 – Kiểm tra firewall local

iptables / nft / ufw

STEP 4 – Kiểm tra routing

- Default gateway phải là 10.10.10.3
- Ping gateway

STEP 5 – Kiểm tra firewall ngoài (quan trọng)

Ở đây ta kiểm tra theo 2 hướng:

5A – Kiểm tra outbound (server → Internet)

Server thử connect ra ngoài:

- nc 192.168.232.1
- traceroute 192.168.232.1

Nếu không đi được → pfSense LAN rule sai

5B – Kiểm tra inbound (Internet → Server qua WAN)

Cách chuẩn trong lab:

1. Bật tcpdump trên server
2. Thực hiện kết nối tới WAN IP (192.168.232.130) từ một máy Internet-side

Server sẽ xem có nhận được SYN không.

Nếu không thấy SYN:
→ NAT / WAN rule pfSense sai

Nếu thấy SYN nhưng không reply:
→ local firewall hoặc service sai

🧠 Phiên bản Tool hoàn chỉnh cho Lab này

Script dưới đây:

- Kiểm tra service
- Kiểm tra port
- Kiểm tra firewall local
- Kiểm tra gateway
- Kiểm tra outbound
- Bắt packet 5 giây
- Traceroute
- Ghi log đầy đủ

```
#!/usr/bin/env python3

import subprocess
import socket
import argparse
import datetime
import sys
import threading
import time

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout + result.stderr

def log_write(logfile, text):
    print(text)
    logfile.write(text + "\n")

# STEP 1
def check_service(service, logfile):
    log_write(logfile, "\n[STEP 1] Checking service status")
    output = run_cmd(f"systemctl is-active {service}")
    log_write(logfile, output.strip())
    return "active" in output

# STEP 2
def check_port(port, protocol, logfile):
```

```
    log_write(logfile, "\n[STEP 2] Checking listening port")
    if protocol == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"
    output = run_cmd(cmd)
    log_write(logfile, output if output else "Port NOT listening")
    return bool(output)

# STEP 3
def check_local_firewall(logfile):
    log_write(logfile, "\n[STEP 3] Checking local firewall (iptables)")
    output = run_cmd("iptables -L -n -v")
    log_write(logfile, output[:1000])

# STEP 4
def check_gateway(logfile):
    log_write(logfile, "\n[STEP 4] Checking routing & gateway")
    route = run_cmd("ip route | grep default")
    log_write(logfile, route)
    ping = run_cmd("ping -c 2 10.10.10.3")
    log_write(logfile, ping)

# STEP 5A
def test_outbound(logfile):
    log_write(logfile, "\n[STEP 5A] Testing outbound connectivity")
    traceroute = run_cmd("traceroute -n 192.168.232.1")
    log_write(logfile, traceroute)

# STEP 5B - Packet capture
def capture_packets(port, logfile):
    log_write(logfile, "\n[STEP 5B] Capturing packets (5 seconds)")
    cmd = f"timeout 5 tcpdump -nn -i any port {port}"
    output = run_cmd(cmd)
    log_write(logfile, output)

def test_public_connect(ip, port, protocol, logfile):
    log_write(logfile, "\n[STEP 5C] Testing connection to WAN IP")
    try:
        if protocol == "tcp":
            sock = socket.create_connection((ip, port), timeout=5)
            sock.close()
            log_write(logfile, "TCP CONNECT SUCCESS")
        elif protocol == "udp":
            sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            sock.sendto(b"test", (ip, port))
            log_write(logfile, "UDP PACKET SENT")
    except Exception as e:
        log_write(logfile, f"Connection FAILED: {e}")
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--service", required=True)
    parser.add_argument("--service-port", type=int, required=True)
    parser.add_argument("--public-port", type=int, required=True)
    parser.add_argument("--wan-ip", required=True)
    parser.add_argument("--protocol", required=True)
    args = parser.parse_args()

    logfile_name =
f"lab_service_check_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"

    with open(logfile_name, "w") as logfile:

        if not check_service(args.service, logfile):
            log_write(logfile, "Service NOT running. STOP.")
            sys.exit(1)

        if not check_port(args.service_port, args.protocol, logfile):
            log_write(logfile, "Port NOT listening. STOP.")
            sys.exit(1)

        check_local_firewall(logfile)
        check_gateway(logfile)
        test_outbound(logfile)

        capture_packets(args.service_port, logfile)
        test_public_connect(args.wan_ip, args.public_port, args.protocol,
logfile)

    print(f"\nLog saved to {logfile_name}")

if __name__ == "__main__":
    main()
```

📌 Cách sử dụng trong lab của bạn

Ví dụ kiểm tra nginx:

```
python3 service_check_lab.py \
  --service nginx \
  --service-port 80 \
  --public-port 8080 \
  --wan-ip 192.168.232.130 \
  --protocol tcp
```

🔦 Cách kiểm tra inbound đúng cách trong lab

Quan trọng:

Bạn nên có một VM khác trong mạng 192.168.232.0/24 để đóng vai Internet.

Từ VM đó:

```
nc 192.168.232.130 8080
```

Trong khi server đang chạy tool để bắt tcpdump.

Nếu:

- Không thấy SYN → pfSense WAN rule/NAT sai
- Thấy SYN nhưng không reply → local firewall hoặc service sai
- Thấy SYN, SYN-ACK → OK

🎯 Kết luận

Với topology này:

- Kiểm tra phải phân biệt outbound và inbound
- Phải có packet capture để xác định chính xác điểm drop
- Không thể chỉ dựa vào connect() từ server

Nếu bạn muốn, tôi có thể viết phiên bản nâng cao:

- Tự động SSH vào pfSense để kiểm tra rule
- Tự động capture cả WAN và LAN interface
- Xuất báo cáo phân tích: “Drop at WAN”, “Drop at LAN”, “Drop at Server”

Bạn muốn tool ở mức lab học tập hay mức kiểm tra sự cố chuyên sâu như NOC?

tôi muốn đầu vào là tất cả các tham số gồm địa chỉ ip, địa chỉ port, tên dịch vụ. Viết log chi tiết, rõ ràng dễ hiểu

Rõ. Tôi sẽ thiết kế lại tool theo đúng yêu cầu:

- **Input đầy đủ**:
  - IP server
  - Port service nội bộ
  - WAN IP firewall
  - Public port NAT
  - Tên service

    - Protocol
- **Log chi tiết, rõ ràng, có phân tích kết luận từng bước**
- **Phù hợp topology lab của bạn (pfSense WAN 192.168.232.130 / LAN 10.10.10.3 / Server 10.10.10.10)**

🎯 Triết lý kiểm tra

Tool sẽ kiểm tra theo chuỗi logic:

```
1. Service state
2. Listening socket
3. Local firewall
4. Routing & Gateway
5. Outbound connectivity
6. Inbound test (WAN IP)
7. Packet capture verification
8. Final assessment
```

Mỗi bước sẽ ghi:

- ✔ PASS
- ✖ FAIL
- ⚠ WARNING
- Giải thích nguyên nhân
- Gợi ý xử lý

🧠 Script hoàn chỉnh (Lab Version – Detailed Logging)

Lưu thành: `service_diagnostic_lab.py`

```
#!/usr/bin/env python3

import subprocess
import socket
import argparse
import datetime
import sys

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout.strip() + result.stderr.strip()

def log(logfile, message):
    print(message)
    logfile.write(message + "\n")
```

```
def section(logfile, title):
    log(logfile, "\n" + "="*70)
    log(logfile, title)
    log(logfile, "="*70)

def check_service(service, logfile):
    section(logfile, "STEP 1 - SERVICE STATUS CHECK")
    output = run_cmd(f"systemctl is-active {service}")
    if "active" in output:
        log(logfile, f"[PASS] Service '{service}' is ACTIVE.")
        return True
    else:
        log(logfile, f"[FAIL] Service '{service}' is NOT running.")
        log(logfile, "        -> Use: systemctl start <service>")
        return False

def check_port(port, protocol, logfile):
    section(logfile, "STEP 2 - LISTENING PORT CHECK")
    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"

    output = run_cmd(cmd)
    if output:
        log(logfile, f"[PASS] Port {port}/{protocol.upper()} is LISTENING.")
        log(logfile, output)
        return True
    else:
        log(logfile, f"[FAIL] Port {port} is NOT listening.")
        log(logfile, "        -> Check service config / bind address.")
        return False

def check_local_firewall(port, logfile):
    section(logfile, "STEP 3 - LOCAL FIREWALL CHECK (iptables)")
    output = run_cmd("iptables -L INPUT -n -v")

    if f"dpt:{port}" in output or "ACCEPT" in output:
        log(logfile, "[INFO] iptables rules detected.")
        log(logfile, output[:1200])
        log(logfile, "[NOTE] Verify ACCEPT rule for this port.")
    else:
        log(logfile, "[INFO] No explicit rule found for this port.")
        log(logfile, "       If default policy is DROP, traffic may be
blocked.")

def check_routing(gateway_ip, logfile):
    section(logfile, "STEP 4 - ROUTING & GATEWAY CHECK")
```

```
    route = run_cmd("ip route | grep default")
    log(logfile, f"[INFO] Default route: {route}")

    ping = run_cmd(f"ping -c 2 {gateway_ip}")
    if "0% packet loss" in ping:
        log(logfile, f"[PASS] Gateway {gateway_ip} reachable.")
    else:
        log(logfile, f"[FAIL] Cannot reach gateway {gateway_ip}.")
        log(logfile, "       -> Check LAN config / cable / pfSense LAN
interface.")

def test_outbound(logfile):
    section(logfile, "STEP 5 - OUTBOUND CONNECTIVITY TEST")

    traceroute = run_cmd("traceroute -n 192.168.232.1")
    log(logfile, traceroute)

def test_inbound(wan_ip, public_port, protocol, logfile):
    section(logfile, "STEP 6 - INBOUND TEST (CONNECT TO WAN IP)")

    try:
        if protocol.lower() == "tcp":
            sock = socket.create_connection((wan_ip, public_port), timeout=5)
            sock.close()
            log(logfile, f"[PASS] Able to connect to {wan_ip}:{public_port}")
            log(logfile, "       -> NAT & WAN rule likely correct.")
        elif protocol.lower() == "udp":
            sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            sock.sendto(b"test", (wan_ip, public_port))
            log(logfile, "[INFO] UDP packet sent (no handshake
confirmation).")
    except Exception as e:
        log(logfile, f"[FAIL] Cannot connect to {wan_ip}:{public_port}")
        log(logfile, f"       Reason: {e}")
        log(logfile, "       -> Check pfSense NAT rule and WAN firewall
rule.")

def capture_packets(port, logfile):
    section(logfile, "STEP 7 - PACKET CAPTURE (5 SECONDS)")
    cmd = f"timeout 5 tcpdump -nn -i any port {port}"
    output = run_cmd(cmd)
    if output:
        log(logfile, "[INFO] Packets observed:")
        log(logfile, output)
    else:
        log(logfile, "[WARNING] No packets captured.")
        log(logfile, "          -> Inbound traffic may not reach server.")
```

```
def final_summary(logfile):
    section(logfile, "FINAL SUMMARY")
    log(logfile, "Review each step above.")
    log(logfile, "If traffic not seen in tcpdump → problem at pfSense.")
    log(logfile, "If SYN seen but no response → local firewall/service
issue.")
    log(logfile, "If everything PASS → service reachable.")

def main():
    parser = argparse.ArgumentParser(description="Lab Service Diagnostic
Tool")
    parser.add_argument("--server-ip", required=True)
    parser.add_argument("--service", required=True)
    parser.add_argument("--service-port", type=int, required=True)
    parser.add_argument("--wan-ip", required=True)
    parser.add_argument("--public-port", type=int, required=True)
    parser.add_argument("--gateway-ip", required=True)
    parser.add_argument("--protocol", required=True)
    args = parser.parse_args()

    log_file_name =
f"diagnostic_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"

    with open(log_file_name, "w") as logfile:

        section(logfile, "SERVICE DIAGNOSTIC REPORT")
        log(logfile, f"Server IP: {args.server_ip}")
        log(logfile, f"Service: {args.service}")
        log(logfile, f"Internal Port: {args.service_port}")
        log(logfile, f"WAN IP: {args.wan_ip}")
        log(logfile, f"Public Port: {args.public_port}")
        log(logfile, f"Protocol: {args.protocol}")

        if not check_service(args.service, logfile):
            sys.exit(1)

        if not check_port(args.service_port, args.protocol, logfile):
            sys.exit(1)

        check_local_firewall(args.service_port, logfile)
        check_routing(args.gateway_ip, logfile)
        test_outbound(logfile)
        capture_packets(args.service_port, logfile)
        test_inbound(args.wan_ip, args.public_port, args.protocol, logfile)
        final_summary(logfile)

    print(f"\nDetailed log saved to: {log_file_name}")

if __name__ == "__main__":
```

```
    main()
```

📌 Ví dụ chạy trong lab của bạn

```
python3 service_diagnostic_lab.py \
  --server-ip 10.10.10.10 \
  --service nginx \
  --service-port 80 \
  --wan-ip 192.168.232.130 \
  --public-port 8080 \
  --gateway-ip 10.10.10.3 \
  --protocol tcp
```

🩹 Kết quả log sẽ rõ như sau:

```
STEP 1 - SERVICE STATUS CHECK
[PASS] Service 'nginx' is ACTIVE.

STEP 2 - LISTENING PORT CHECK
[PASS] Port 80/TCP is LISTENING.

STEP 6 - INBOUND TEST
[FAIL] Cannot connect to 192.168.232.130:8080
Reason: Connection refused
-> Check pfSense NAT rule and WAN firewall rule.
```

🎯 Ưu điểm phiên bản này

- Log rất rõ ràng
- Dễ đọc cho người học mạng
- Phân tích từng tầng
- Phù hợp lab pfSense của bạn
- Dễ mở rộng thêm JSON / HTML report

Nếu bạn muốn, tôi có thể nâng cấp thêm:

- 🔎 Tự phân tích log và kết luận “Drop tại WAN”
- 📊 Xuất báo cáo HTML
- 🔐 Tự SSH vào pfSense kiểm tra NAT rule
- 🚀 Version đa luồng kiểm tra nhiều service cùng lúc

Bạn muốn nâng cấp theo hướng học tập hay xây dựng thành tool demo đồ án?

tại máy server vì nằm trong mạng lan nên sẽ không thể truy cập được tới interface wan của firewall nên không thể kiểm tra inbound, khi bắt gói tin sử dụng các lệnh sudo để được cấp quyền đầy đủ

Chính xác.

Trong topology của bạn:

```
Server (10.10.10.10)
        |
LAN 10.10.10.0/24
        |
pfSense LAN: 10.10.10.3
pfSense WAN: 192.168.232.130
```

Server **không thể kiểm tra inbound bằng cách tự connect tới WAN IP** vì:

- Hairpin NAT có thể bị disable
- Route không quay lại đúng
- Không phản ánh đúng Internet inbound thật

Vì vậy:

☞ Tool trên server **không được phép giả lập inbound bằng connect tới WAN IP**
☞ Thay vào đó phải:

1. Bắt gói tin trên server (sudo tcpdump)
2. Chờ kết nối từ Internet-side VM
3. Phân tích có nhận được SYN hay không

🎯 Thiết kế lại đúng chuẩn mạng

Tool trên server sẽ:

✔ Kiểm tra nội bộ

- Service
- Port
- Local firewall
- Route

✔ Kiểm tra outbound

✔ Bắt packet inbound bằng sudo

❌ KHÔNG tự connect tới WAN IP

Inbound test phải thực hiện từ máy Internet-side.

🧠 Phiên bản Tool chuẩn cho kiến trúc này

Lưu file: `service_diagnostic_lan_server.py`

```
#!/usr/bin/env python3

import subprocess
import argparse
import datetime
import sys
import os

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout.strip() + result.stderr.strip()

def log(logfile, msg):
    print(msg)
    logfile.write(msg + "\n")

def section(logfile, title):
    log(logfile, "\n" + "="*75)
    log(logfile, title)
    log(logfile, "="*75)

def check_service(service, logfile):
    section(logfile, "STEP 1 - SERVICE STATUS CHECK")

    output = run_cmd(f"systemctl is-active {service}")

    if "active" in output:
        log(logfile, f"[PASS] Service '{service}' is ACTIVE.")
        return True
    else:
        log(logfile, f"[FAIL] Service '{service}' is NOT running.")
        return False

def check_listening(port, protocol, logfile):
    section(logfile, "STEP 2 - LISTENING SOCKET CHECK")
```

```
    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"

    output = run_cmd(cmd)

    if output:
        log(logfile, f"[PASS] Port {port}/{protocol.upper()} is LISTENING.")
        log(logfile, output)
        return True
    else:
        log(logfile, f"[FAIL] Port {port} is NOT listening.")
        return False

def check_local_firewall(port, logfile):
    section(logfile, "STEP 3 - LOCAL FIREWALL CHECK (iptables)")

    output = run_cmd("sudo iptables -L INPUT -n -v")

    log(logfile, output[:1500])

    if "DROP" in output:
        log(logfile, "[WARNING] DROP policy detected. Verify ACCEPT rule for
this port.")

def check_routing(gateway, logfile):
    section(logfile, "STEP 4 - ROUTING & GATEWAY CHECK")

    route = run_cmd("ip route | grep default")
    log(logfile, f"[INFO] Default route: {route}")

    ping = run_cmd(f"ping -c 2 {gateway}")

    if "0% packet loss" in ping:
        log(logfile, f"[PASS] Gateway {gateway} reachable.")
    else:
        log(logfile, f"[FAIL] Cannot reach gateway {gateway}.")

def test_outbound(test_ip, logfile):
    section(logfile, "STEP 5 - OUTBOUND CONNECTIVITY TEST")

    traceroute = run_cmd(f"traceroute -n {test_ip}")
    log(logfile, traceroute)

def capture_inbound(port, protocol, duration, logfile):
    section(logfile, "STEP 6 - INBOUND PACKET CAPTURE")

    log(logfile, f"[INFO] Capturing {protocol.upper()} traffic on port {port}
```

```
for {duration} seconds.")
    log(logfile, "[INFO] During this time, initiate connection from Internet-
side VM.")

    if protocol.lower() == "tcp":
        filter_exp = f"tcp port {port}"
    elif protocol.lower() == "udp":
        filter_exp = f"udp port {port}"
    else:
        filter_exp = "icmp"

    cmd = f"sudo timeout {duration} tcpdump -nn -i any {filter_exp}"

    output = run_cmd(cmd)

    if output:
        log(logfile, "\n[INFO] Packets observed:")
        log(logfile, output)

        if "Flags [S]" in output:
            log(logfile, "\n[PASS] SYN packet received.")
            log(logfile, "       -> Traffic successfully reached server.")
        else:
            log(logfile, "\n[INFO] Packets seen but no SYN flag detected.")
    else:
        log(logfile, "\n[FAIL] No inbound packets captured.")
        log(logfile, "       -> Traffic likely blocked at pfSense WAN or
NAT.")

def final_summary(logfile):
    section(logfile, "FINAL ANALYSIS GUIDE")

    log(logfile, "If no packets captured:")
    log(logfile, "   -> Check pfSense WAN firewall rule.")
    log(logfile, "   -> Check NAT port forward.")
    log(logfile, "")
    log(logfile, "If SYN received but no response:")
    log(logfile, "   -> Check local firewall.")
    log(logfile, "   -> Check service bind address.")
    log(logfile, "")
    log(logfile, "If SYN and SYN-ACK seen:")
    log(logfile, "   -> Service reachable from Internet.")

def main():
    parser = argparse.ArgumentParser(description="LAN Server Service
Diagnostic Tool")
    parser.add_argument("--service", required=True)
    parser.add_argument("--service-port", type=int, required=True)
    parser.add_argument("--gateway-ip", required=True)
```

```
    parser.add_argument("--test-outbound-ip", required=True)
    parser.add_argument("--protocol", required=True)
    parser.add_argument("--capture-time", type=int, default=5)

    args = parser.parse_args()

    logfile_name =
f"lan_diagnostic_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"

    with open(logfile_name, "w") as logfile:

        section(logfile, "LAN SERVER SERVICE DIAGNOSTIC REPORT")

        if not check_service(args.service, logfile):
            sys.exit(1)

        if not check_listening(args.service_port, args.protocol, logfile):
            sys.exit(1)

        check_local_firewall(args.service_port, logfile)
        check_routing(args.gateway_ip, logfile)
        test_outbound(args.test_outbound_ip, logfile)
        capture_inbound(args.service_port, args.protocol, args.capture_time,
logfile)
        final_summary(logfile)

    print(f"\nLog saved to: {logfile_name}")

if __name__ == "__main__":
    main()
```

📌 Cách sử dụng trong lab của bạn

Ví dụ kiểm tra nginx:

```
python3 service_diagnostic_lan_server.py \
  --service nginx \
  --service-port 80 \
  --gateway-ip 10.10.10.3 \
  --test-outbound-ip 192.168.232.1 \
  --protocol tcp \
  --capture-time 10
```

Sau đó từ VM Internet-side:

```
nc 192.168.232.130 8080
```

🎯 Kết quả phân tích sẽ chính xác

Kết quả tcpdump	Ý nghĩa
Không thấy gói	Bị block tại WAN/NAT
Thấy SYN	Đã qua firewall
Thấy SYN + SYN-ACK	Service OK
Thấy SYN nhưng không SYN-ACK	Local firewall hoặc service lỗi

🔐 Lưu ý quan trọng

Vì dùng `sudo tcpdump` và `sudo iptables`:

Bạn nên:

```
sudo chmod +x service_diagnostic_lan_server.py
sudo setcap cap_net_raw,cap_net_admin=eip $(which tcpdump)
```

Hoặc chạy script bằng sudo:

```
sudo python3 service_diagnostic_lan_server.py ...
```

📌 Kết luận kiến trúc

Trong mô hình LAN server sau firewall:

- Server KHÔNG thể tự kiểm tra inbound WAN
- Phải dùng packet capture
- Inbound test phải thực hiện từ Internet-side
- Tool chỉ phân tích và ghi log

Nếu bạn muốn, tôi có thể viết thêm:

- Phiên bản 2 phần (Client-side + Server-side)
- Client sẽ tự gửi traffic và server tự phân tích
- Hoặc version tự động phân tích pcap và đưa kết luận chi tiết hơn

Bạn muốn dừng ở mức lab học tập hay làm thành tool chuẩn để demo chuyên đề mạng?

sử dụng dịch vụ ufw thay thế cho iptable

Tốt. Ta sẽ chỉnh lại tool để:

- ❌ Không dùng iptables trực tiếp
- ✅ Sử dụng **UFW (Uncomplicated Firewall)**

- ✅ Phân tích rule rõ ràng
- ✅ Log dễ hiểu
- ✅ Phù hợp môi trường LAN server sau pfSense
- ✅ Dùng sudo cho các thao tác cần quyền root

🎯 Cách kiểm tra UFW đúng kỹ thuật

Trong Linux dùng UFW, ta phải kiểm tra:

1. Trạng thái UFW
2. Default policy (Incoming / Outgoing)
3. Rule cho port cụ thể
4. Logging của UFW

Lệnh cần dùng:

```
sudo ufw status verbose
sudo ufw status numbered
```

🧠 Logic phân tích UFW

Ví dụ output:

```
Status: active

Default: deny (incoming), allow (outgoing)
To                         Action      From
80/tcp                     ALLOW       Anywhere
```

Phân tích:

Trường hợp	Kết luận
UFW inactive	Không chặn
Default deny + không có allow rule	Bị chặn
Có ALLOW rule	OK
Có DENY rule	Bị chặn

🚀 Phiên bản Tool dùng UFW (Production Lab Version)

Lưu file: `service_diagnostic_ufw.py`

```python
#!/usr/bin/env python3

import subprocess
import argparse
import datetime
import sys

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout.strip() + result.stderr.strip()

def log(logfile, msg):
    print(msg)
    logfile.write(msg + "\n")

def section(logfile, title):
    log(logfile, "\n" + "="*75)
    log(logfile, title)
    log(logfile, "="*75)

# STEP 1
def check_service(service, logfile):
    section(logfile, "STEP 1 - SERVICE STATUS CHECK")
    output = run_cmd(f"systemctl is-active {service}")

    if "active" in output:
        log(logfile, f"[PASS] Service '{service}' is ACTIVE.")
        return True
    else:
        log(logfile, f"[FAIL] Service '{service}' is NOT running.")
        return False

# STEP 2
def check_listening(port, protocol, logfile):
    section(logfile, "STEP 2 - LISTENING SOCKET CHECK")

    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"

    output = run_cmd(cmd)

    if output:
        log(logfile, f"[PASS] Port {port}/{protocol.upper()} is LISTENING.")
        log(logfile, output)
        return True
    else:
        log(logfile, f"[FAIL] Port {port} is NOT listening.")
```

```
        return False

# STEP 3 - UFW
def check_ufw(port, protocol, logfile):
    section(logfile, "STEP 3 - UFW FIREWALL CHECK")

    status = run_cmd("sudo ufw status verbose")

    if "inactive" in status:
        log(logfile, "[INFO] UFW is INACTIVE.")
        log(logfile, "       -> No local firewall filtering by UFW.")
        return

    log(logfile, "[INFO] UFW is ACTIVE.")
    log(logfile, status)

    if f"{port}/{protocol.lower()}" in status and "ALLOW" in status:
        log(logfile, f"[PASS] UFW allows {port}/{protocol.upper()}.")
    elif f"{port}/{protocol.lower()}" in status and "DENY" in status:
        log(logfile, f"[FAIL] UFW explicitly DENIES
{port}/{protocol.upper()}.")
    else:
        if "deny (incoming)" in status.lower():
            log(logfile, f"[WARNING] Default incoming policy is DENY.")
            log(logfile, f"          No explicit ALLOW rule for
{port}/{protocol.upper()}.")
            log(logfile, f"          -> Traffic will be BLOCKED.")
        else:
            log(logfile, "[INFO] Default incoming policy is ALLOW.")

# STEP 4
def check_routing(gateway, logfile):
    section(logfile, "STEP 4 - ROUTING & GATEWAY CHECK")

    route = run_cmd("ip route | grep default")
    log(logfile, f"[INFO] Default route: {route}")

    ping = run_cmd(f"ping -c 2 {gateway}")

    if "0% packet loss" in ping:
        log(logfile, f"[PASS] Gateway {gateway} reachable.")
    else:
        log(logfile, f"[FAIL] Cannot reach gateway {gateway}.")

# STEP 5
def test_outbound(test_ip, logfile):
    section(logfile, "STEP 5 - OUTBOUND CONNECTIVITY TEST")

    traceroute = run_cmd(f"traceroute -n {test_ip}")
```

```
    log(logfile, traceroute)

# STEP 6
def capture_inbound(port, protocol, duration, logfile):
    section(logfile, "STEP 6 - INBOUND PACKET CAPTURE")

    log(logfile, f"[INFO] Capturing {protocol.upper()} traffic on port {port}
for {duration} seconds.")
    log(logfile, "[INFO] Initiate connection from Internet-side VM now.")

    if protocol.lower() == "tcp":
        filter_exp = f"tcp port {port}"
    elif protocol.lower() == "udp":
        filter_exp = f"udp port {port}"
    else:
        filter_exp = "icmp"

    cmd = f"sudo timeout {duration} tcpdump -nn -i any {filter_exp}"

    output = run_cmd(cmd)

    if output:
        log(logfile, "\n[INFO] Packets observed:")
        log(logfile, output)

        if "Flags [S]" in output:
            log(logfile, "\n[PASS] SYN packet received.")
            log(logfile, "       -> Traffic successfully reached server.")
        else:
            log(logfile, "\n[INFO] Packets seen but no SYN flag detected.")
    else:
        log(logfile, "\n[FAIL] No inbound packets captured.")
        log(logfile, "       -> Likely blocked at pfSense WAN or NAT.")

def final_summary(logfile):
    section(logfile, "FINAL DIAGNOSTIC SUMMARY")

    log(logfile, "Interpretation Guide:")
    log(logfile, "- No packet captured  -> Problem at pfSense WAN/NAT.")
    log(logfile, "- Packet captured but no response -> UFW or service
issue.")
    log(logfile, "- SYN + SYN-ACK seen -> Service reachable from Internet.")

def main():
    parser = argparse.ArgumentParser(description="Service Diagnostic Tool
(UFW Version)")
    parser.add_argument("--service", required=True)
    parser.add_argument("--service-port", type=int, required=True)
    parser.add_argument("--gateway-ip", required=True)
```

```
    parser.add_argument("--test-outbound-ip", required=True)
    parser.add_argument("--protocol", required=True)
    parser.add_argument("--capture-time", type=int, default=5)

    args = parser.parse_args()

    logfile_name =
f"ufw_diagnostic_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"

    with open(logfile_name, "w") as logfile:

        section(logfile, "SERVICE DIAGNOSTIC REPORT (UFW VERSION)")

        if not check_service(args.service, logfile):
            sys.exit(1)

        if not check_listening(args.service_port, args.protocol, logfile):
            sys.exit(1)

        check_ufw(args.service_port, args.protocol, logfile)
        check_routing(args.gateway_ip, logfile)
        test_outbound(args.test_outbound_ip, logfile)
        capture_inbound(args.service_port, args.protocol, args.capture_time,
logfile)
        final_summary(logfile)

    print(f"\nDetailed log saved to: {logfile_name}")

if __name__ == "__main__":
    main()
```

📌 Cách chạy trong lab của bạn

```
sudo python3 service_diagnostic_ufw.py \
  --service nginx \
  --service-port 80 \
  --gateway-ip 10.10.10.3 \
  --test-outbound-ip 192.168.232.1 \
  --protocol tcp \
  --capture-time 10
```

🎯 Phân tích kết quả sẽ rõ ràng

Ví dụ log:

```
STEP 3 - UFW FIREWALL CHECK
[INFO] UFW is ACTIVE.
Default: deny (incoming), allow (outgoing)
```

```
[WARNING] Default incoming policy is DENY.
No explicit ALLOW rule for 80/TCP.
-> Traffic will be BLOCKED.
```

→ Bạn biết ngay phải:

```
sudo ufw allow 80/tcp
```

🔐 Lưu ý quan trọng

Chạy script bằng sudo vì:

- ufw status verbose cần root
- tcpdump cần quyền raw socket

🎓 Kết luận kỹ thuật

Phiên bản này:

- Phù hợp lab pfSense của bạn
- Không cố gắng hairpin NAT sai logic
- Phân tích rõ inbound/outbound
- Dùng UFW thay iptables
- Log chi tiết dễ hiểu

Nếu bạn muốn, tôi có thể nâng cấp thêm:

- Tự parse rule UFW chi tiết theo số thứ tự
- Tự động phát hiện default policy và đưa ra kết luận chính xác 100%
- Xuất file JSON để làm báo cáo đồ án

Bạn muốn nâng cấp mức nào?

Viết lại tool trên dưới dạng script Python và có thể tùy chỉnh cho các dịch vụ khác nhau. Các bước kiểm tra dịch vụ gồm 1. Kiểm tra trạng thái dịch vụ tại máy chủ - sử dụng sudo systemctl status để kiểm tra 2. Kiểm tra port service - sử dụng ss + grep để kiểm tra port 3. kiểm tra firewall local trong server - sử dụng ufw để kiểm tra xem đã mở port dịch vụ chưa 4. kiểm tra firewall bên ngoài - ping đến interface wan của fw để kiểm tra xem đã có thể đi qua firewall chưa 5. Test dịch vụ có tới được người dùng ở internet hay không - sử dụng ping 8.8.8.8 để kiểm tra kết nối internet

Dưới đây là phiên bản **script Python hoàn chỉnh**, có thể tái sử dụng cho nhiều dịch vụ khác nhau, đúng theo 5 bước bạn yêu cầu và phù hợp mô hình server nằm trong LAN sau firewall.

Script:

- Sử dụng `sudo systemctl status`
- Sử dụng `ss + grep`
- Kiểm tra `ufw`
- Ping WAN firewall
- Ping 8.8.8.8 kiểm tra Internet
- Ghi log chi tiết, rõ ràng, dễ đọc
- Có phân tích PASS / FAIL

📌 File: `service_check_full.py`

```
#!/usr/bin/env python3

import subprocess
import argparse
import datetime
import sys

# ========================
# Utility Functions
# ========================

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout + result.stderr

def log(logfile, message):
    print(message)
    logfile.write(message + "\n")

def section(logfile, title):
    log(logfile, "\n" + "="*80)
    log(logfile, title)
    log(logfile, "="*80)

# ========================
# STEP 1 - Service Status
# ========================

def check_service(service, logfile):
    section(logfile, "STEP 1 - CHECK SERVICE STATUS")

    cmd = f"sudo systemctl status {service}"
    output = run_cmd(cmd)

    log(logfile, f"Command: {cmd}")
    log(logfile, "-"*60)
    log(logfile, output)
```

```
    if "Active: active (running)" in output:
        log(logfile, "[PASS] Service is running normally.")
        return True
    else:
        log(logfile, "[FAIL] Service is NOT running.")
        return False

# ========================
# STEP 2 - Check Listening Port
# ========================

def check_port(port, protocol, logfile):
    section(logfile, "STEP 2 - CHECK LISTENING PORT")

    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"

    output = run_cmd(cmd)

    log(logfile, f"Command: {cmd}")
    log(logfile, "-"*60)

    if output.strip():
        log(logfile, output)
        log(logfile, "[PASS] Port is LISTENING.")
        return True
    else:
        log(logfile, "[FAIL] Port is NOT listening.")
        return False

# ========================
# STEP 3 - Check UFW
# ========================

def check_ufw(port, protocol, logfile):
    section(logfile, "STEP 3 - CHECK LOCAL FIREWALL (UFW)")

    cmd = "sudo ufw status verbose"
    output = run_cmd(cmd)

    log(logfile, f"Command: {cmd}")
    log(logfile, "-"*60)
    log(logfile, output)

    if "inactive" in output.lower():
        log(logfile, "[INFO] UFW is inactive. No local firewall blocking.")
```

```
        return True

    rule_string = f"{port}/{protocol.lower()}"

    if rule_string in output and "ALLOW" in output:
        log(logfile, f"[PASS] UFW allows {rule_string}.")
        return True
    else:
        log(logfile, f"[WARNING] No explicit ALLOW rule for {rule_string}.")
        log(logfile, "Check default incoming policy.")
        return False

# ==========================
# STEP 4 - Check Firewall WAN Reachability
# ==========================

def check_firewall_wan(wan_ip, logfile):
    section(logfile, "STEP 4 - CHECK FIREWALL WAN REACHABILITY")

    cmd = f"ping -c 3 {wan_ip}"
    output = run_cmd(cmd)

    log(logfile, f"Command: {cmd}")
    log(logfile, "-"*60)
    log(logfile, output)

    if "0% packet loss" in output:
        log(logfile, "[PASS] WAN interface reachable.")
        return True
    else:
        log(logfile, "[FAIL] Cannot reach WAN interface.")
        log(logfile, "Possible routing or firewall issue.")
        return False

# ==========================
# STEP 5 - Check Internet Connectivity
# ==========================

def check_internet(logfile):
    section(logfile, "STEP 5 - CHECK INTERNET CONNECTIVITY")

    cmd = "ping -c 3 8.8.8.8"
    output = run_cmd(cmd)

    log(logfile, f"Command: {cmd}")
    log(logfile, "-"*60)
    log(logfile, output)

    if "0% packet loss" in output:
```

```
        log(logfile, "[PASS] Internet connectivity OK.")
        return True
    else:
        log(logfile, "[FAIL] No Internet connectivity.")
        return False

# ========================
# Main
# ========================

def main():
    parser = argparse.ArgumentParser(description="Service Diagnostic Tool")

    parser.add_argument("--service", required=True, help="Service name")
    parser.add_argument("--port", type=int, required=True, help="Service
port")
    parser.add_argument("--protocol", required=True, help="tcp/udp")
    parser.add_argument("--wan-ip", required=True, help="Firewall WAN IP")

    args = parser.parse_args()

    log_filename =
f"service_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.log"

    with open(log_filename, "w") as logfile:

        section(logfile, "SERVICE DIAGNOSTIC REPORT")
        log(logfile, f"Service Name : {args.service}")
        log(logfile, f"Service Port : {args.port}")
        log(logfile, f"Protocol     : {args.protocol}")
        log(logfile, f"Firewall WAN : {args.wan_ip}")

        service_ok = check_service(args.service, logfile)
        port_ok = check_port(args.port, args.protocol, logfile)
        ufw_ok = check_ufw(args.port, args.protocol, logfile)
        wan_ok = check_firewall_wan(args.wan_ip, logfile)
        internet_ok = check_internet(logfile)

        section(logfile, "FINAL SUMMARY")

        if all([service_ok, port_ok, ufw_ok, wan_ok, internet_ok]):
            log(logfile, "[RESULT] Service environment looks healthy.")
        else:
            log(logfile, "[RESULT] One or more checks FAILED. Review above
steps.")

    print(f"\nReport saved to: {log_filename}")

if __name__ == "__main__":
```

```
    main()
```

📌 Cách sử dụng

Ví dụ kiểm tra nginx:

```
sudo python3 service_check_full.py \
  --service nginx \
  --port 80 \
  --protocol tcp \
  --wan-ip 192.168.232.130
```

🎯 Kết quả log sẽ rõ ràng như:

```
STEP 1 - CHECK SERVICE STATUS
[PASS] Service is running normally.

STEP 2 - CHECK LISTENING PORT
[PASS] Port is LISTENING.

STEP 3 - CHECK LOCAL FIREWALL (UFW)
[PASS] UFW allows 80/tcp.

STEP 4 - CHECK FIREWALL WAN REACHABILITY
[PASS] WAN interface reachable.

STEP 5 - CHECK INTERNET CONNECTIVITY
[PASS] Internet connectivity OK.
```

⚠ Lưu ý kỹ thuật

- Phải chạy script bằng `sudo`
- Ping WAN chỉ kiểm tra layer 3 (không chứng minh NAT inbound hoạt động)
- Ping 8.8.8.8 chỉ kiểm tra outbound connectivity

Nếu bạn muốn, tôi có thể nâng cấp:

- Thêm traceroute phân tích đường đi
- Thêm tcpdump kiểm tra inbound thực sự
- Xuất báo cáo JSON/HTML
- Tự động phân tích và đưa ra chẩn đoán chính xác hơn

Bạn muốn nâng cấp theo hướng lab học tập hay xây dựng thành tool demo chuyên đề mạng?

Nâng cấp tự động phân tích và đưa ra chẩn đoán chính xác hơn

Tốt. Tôi sẽ nâng cấp tool theo hướng:

- Không chỉ PASS/FAIL
- Mà còn **tự động suy luận nguyên nhân**
- Đưa ra **chẩn đoán kỹ thuật cụ thể**
- Gợi ý xử lý tương ứng
- Phân tích theo logic mạng nhiều tầng

🎯 Logic phân tích nâng cao

Tool sẽ phân loại lỗi theo ma trận:

Service	Port	UFW	WAN	Internet	Kết luận
FAIL	-	-	-	-	Service down
PASS	FAIL	-	-	-	Service không bind
PASS	PASS	FAIL	-	-	Bị block local firewall
PASS	PASS	PASS	FAIL	PASS	Routing / WAN rule
PASS	PASS	PASS	PASS	FAIL	Outbound bị chặn
PASS	PASS	PASS	PASS	PASS	Môi trường OK

🚀 Phiên bản nâng cấp: `service_diagnostic_pro.py`

Script này:

- Phân tích kết quả
- Đưa ra nguyên nhân chính
- Đưa khuyến nghị cụ thể

```
#!/usr/bin/env python3

import subprocess
import argparse
import datetime
import sys

def run_cmd(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    return result.stdout + result.stderr

def log(logfile, msg):
    print(msg)
    logfile.write(msg + "\n")

def section(logfile, title):
```

```
    log(logfile, "\n" + "="*85)
    log(logfile, title)
    log(logfile, "="*85)

# ========================
# CHECK FUNCTIONS
# ========================

def check_service(service):
    output = run_cmd(f"sudo systemctl status {service}")
    return "Active: active (running)" in output, output

def check_port(port, protocol):
    if protocol.lower() == "tcp":
        cmd = f"ss -lntp | grep :{port}"
    else:
        cmd = f"ss -lnup | grep :{port}"
    output = run_cmd(cmd)
    return bool(output.strip()), output

def check_ufw(port, protocol):
    output = run_cmd("sudo ufw status verbose")
    if "inactive" in output.lower():
        return True, output
    rule = f"{port}/{protocol.lower()}"
    if rule in output and "ALLOW" in output:
        return True, output
    return False, output

def check_ping(target):
    output = run_cmd(f"ping -c 3 {target}")
    return "0% packet loss" in output, output

# ========================
# DIAGNOSTIC ENGINE
# ========================

def analyze(results):
    service_ok = results["service"]
    port_ok = results["port"]
    ufw_ok = results["ufw"]
    wan_ok = results["wan"]
    internet_ok = results["internet"]

    if not service_ok:
        return "CRITICAL: Service is not running.", \
               "Start the service using: sudo systemctl start <service>"

    if service_ok and not port_ok:
```

```
        return "ERROR: Service running but port not listening.", \
               "Check bind address or service configuration."

    if service_ok and port_ok and not ufw_ok:
        return "ERROR: Local firewall (UFW) blocking the port.", \
               "Run: sudo ufw allow <port>/<protocol>"

    if service_ok and port_ok and ufw_ok and not wan_ok:
        return "ERROR: Cannot reach Firewall WAN interface.", \
               "Check routing table and pfSense rules."

    if service_ok and port_ok and ufw_ok and wan_ok and not internet_ok:
        return "WARNING: No Internet connectivity.", \
               "Check pfSense outbound NAT or WAN gateway."

    if all([service_ok, port_ok, ufw_ok, wan_ok, internet_ok]):
        return "SUCCESS: All checks passed.", \
               "Environment appears healthy."

    return "UNKNOWN STATE", "Manual investigation required."

# ========================
# MAIN
# ========================

def main():
    parser = argparse.ArgumentParser(description="Advanced Service Diagnostic
Tool")

    parser.add_argument("--service", required=True)
    parser.add_argument("--port", type=int, required=True)
    parser.add_argument("--protocol", required=True)
    parser.add_argument("--wan-ip", required=True)

    args = parser.parse_args()

    logfile_name =
f"service_diagnostic_pro_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.
log"

    with open(logfile_name, "w") as logfile:

        section(logfile, "ADVANCED SERVICE DIAGNOSTIC REPORT")

        # Perform checks
        service_ok, service_output = check_service(args.service)
        port_ok, port_output = check_port(args.port, args.protocol)
        ufw_ok, ufw_output = check_ufw(args.port, args.protocol)
        wan_ok, wan_output = check_ping(args.wan_ip)
```

```
        internet_ok, internet_output = check_ping("8.8.8.8")

        # Log raw outputs
        section(logfile, "RAW CHECK OUTPUTS")

        log(logfile, "[Service Status]")
        log(logfile, service_output)

        log(logfile, "[Port Check]")
        log(logfile, port_output)

        log(logfile, "[UFW Status]")
        log(logfile, ufw_output)

        log(logfile, "[WAN Ping]")
        log(logfile, wan_output)

        log(logfile, "[Internet Ping]")
        log(logfile, internet_output)

        # Analysis
        section(logfile, "AUTOMATED DIAGNOSTIC RESULT")

        results = {
            "service": service_ok,
            "port": port_ok,
            "ufw": ufw_ok,
            "wan": wan_ok,
            "internet": internet_ok
        }

        diagnosis, recommendation = analyze(results)

        log(logfile, f"Diagnosis      : {diagnosis}")
        log(logfile, f"Recommendation : {recommendation}")

    print(f"\nDetailed report saved to: {logfile_name}")

if __name__ == "__main__":
    main()
```

📌 Cách chạy

```
sudo python3 service_diagnostic_pro.py \
  --service nginx \
  --port 80 \
  --protocol tcp \
  --wan-ip 192.168.232.130
```

🎯 Ví dụ kết quả thông minh

Trường hợp UFW block:

```
Diagnosis      : ERROR: Local firewall (UFW) blocking the port.
Recommendation : Run: sudo ufw allow 80/tcp
```

Trường hợp service down:

```
Diagnosis      : CRITICAL: Service is not running.
Recommendation : Start the service using: sudo systemctl start <service>
```

Trường hợp mọi thứ OK:

```
Diagnosis      : SUCCESS: All checks passed.
Recommendation : Environment appears healthy.
```

🚀 Nâng cấp tiếp theo có thể làm

- Phân tích thêm traceroute
- Phát hiện default route sai gateway
- Phân tích DNS resolution
- Xuất JSON cho báo cáo đồ án
- Tự động vẽ sơ đồ lỗi

Nếu bạn muốn, tôi có thể viết phiên bản:

- 🔎 Chẩn đoán theo mô hình OSI (Layer 3/4/7)
- 📊 Xuất báo cáo HTML chuyên nghiệp
- 🧠 Hệ thống scoring mức độ lỗi (Critical / Major / Minor)

Bạn muốn nâng cấp theo hướng học thuật (đồ án) hay theo hướng công cụ vận hành thực tế?